

Structure Of Operating System

Where does an operating system resides:

An Operating System (OS) resides in memory, but different parts of it live in different places depending on their purpose.

The Kernel always resides in the RAM while the OS is running after its loaded by the bootloader, some drivers and OS files are loaded into the RAM when they are active, Before the OS starts, most of the OS (kernel image, drivers, services) resides on the disk (SSD/HDD).

Layer Model of OS:

Why divide it into layers:

we divide OS into layers to provide structure, make it easier to understand and develop, and make it more organized.

This makes it easier to understand, you don't need to understand the entire OS since each layer focus on a certain task.

How Layers Work together:

Each layer communicated directly only with its adjacent layer so communication goes one step up or one step down layer 4 can never communicate with layer 1 directly. This takes place to prevent chaos.

Layer Communication:

Layers interact with rules and protocols using interfaces, the protocols defines what request are allowed and what not and what data can be exchanged.

This keeps the system stable, and prevent errors.

Idea of OS layered Model:

The Layer Model is a conceptual way to understand how an Operating System (OS) is organized internally.

It divides the OS into layers, where each layer has a specific purpose and level of abstraction.

- Outer layers depend on inner layers
- Inner layers do NOT depend on outer layers
- Requests always flow *down* the layers
- Results flow *up* the layers

Minimal 3 Layers as simplified concept

To simplify the idea, we often describe an OS using **3 basic layers**:

Layer 1: Hardware Layer (Lowest Level)

- CPU
- RAM
- Disks
- Hardware controllers

Layer 2: Kernel/System Layer (Middle Level)

- Memory management
- Device drivers
- CPU scheduling
- File systems
- System calls (API for upper layer)

Layer 3: User Layer (Highest Level)

- Operating system services
- User applications
- Shell / GUI

Hardware dependent part of the OS:

Converts hardware signals into OS operations and vice versa, accesses and controls the physical memory and other hardware like keyboard. Includes the drivers.

Examples:

Basic IO Services:

Provides general OS services (functions like read and write) in a very standardized way that works the same no matter what hardware you use.

Offers standardized operations to higher layers so if the application changes it still can open camera using the standardized functions it still can read and write even if the hardware has changed for the same OS and the same app.

Application Interface and user Interface:

Provides the environment where programs run.

Offers user-friendly ways to interact with the OS.

It has the GUI menu, icons and so on.

It's the most abstract layer, it has the windows libraries files.

OS 5 modeled layers:**Kernel Hardware dependent:**

This is the part that actually talks to the hardware and translates OS command into hardware commands aka signals. Handles the physical memory and addresses.

Kernel Hardware Independent:

This layer does not depend on the type of the hardware it has functions that works on all different types of hardware.

it manages memory management, and processes scheduling

Standard library:

A library that applications use to communicate with the kernel, it has commands and functions like (open, read, write, save, delete). Turns programming functions into system calls.

Shell bash and cmd:

This is the program that the user actually runs,

The user: the human using and interacting with the system

Interfaces:

Interface A between kernel HW dependent and HW:

Connect physical drivers to actual hardware. Drivers use this interface to talk directly to the hardware

Interface B between kernel HW Independent and kernel HW Dependent:

Lets the upper kernel functions (scheduling, memory mgmt, file system) work without caring about what hardware exists

Interface C between HW independent Kernel and OS lib:

This is how programs ask the OS to do things

Allows user programs to request OS services

Uses syscalls like:

- open()
- read()
- write()
- fork()
- exec()

Interface D between OS lib and bash and CLI:

Apps use this interface to call functions to execute it

Interface E between user and bash:

User uses this interface to communicate with the OS bash

Allows the human user to interact with the system

User Mode Vs Kernel Mode

User Mode (User space):

The mode which applications normally runs, when a user wants to execute a function or a process the OS switches to Kernel mode until it finishes the task then goes back to the user mode, this prevents any system crashes.

It cannot access kernel memory or hardware directly.

When it needs to finish a task like a logical task it has to do a system call first.

Kernel Mode (Privileged mode):

The mode where OS kernel runs has full access to hardware and memory.

System Calls

User has their memory, but they cannot access other memory that are not allocated for them and they cannot access hardware. when they need to do a task they either make a system call or they use the standard lib which makes a system call for them. the system then make something called context switch it takes this command from the application and switch to kernel mode executes it and then goes back to the user mode.

Library function call (e.g. fopen, fread, fwrite, fseek)

When we make a direct system call we directly have a Syscall function when we make a wrapper function or a glibc (general library call) we call the function by its name directly getpid()

Feature	System Call	Library Function (Wrapper)
Abstraction	Low — talks directly to the kernel	High — hides system call details
Portability	Low — depends on OS and hardware	High — works across systems using the library
Error Handling	Kernel returns raw errors	Library provides consistent error handling
Ease of Use	Harder — programmer must know syscall numbers	Easier — programmer just calls a function like getpid()

Privileges on x86 CPU - Family

x86 CPUs have 4 privilege levels, stored as 2 bits:

- 0 (highest) – Kernel / OS
- 1 – (rarely used)
- 2 – (rarely used)
- 3 (lowest) – User processes

Most OSes use only:

- 0 for kernel
- 3 for user programs
because other CPU architectures only have 2 levels.

These levels determine what instructions code is allowed to run.

The CS (Code Segment) register contains:

- The memory segment where the current code runs
- 2 bits representing the Current Privilege Level (CPL)

The CPL tells the CPU:

- “Is this code kernel-level (privileged)?” → CPL 0
- “Is this code user-level (restricted)?” → CPL 3

The CPU checks this before executing risky operations.

Privileges on AArch64 CPUs:

EL stands for exception levels

EL0 User applications Restricted

EL1 Operating System kernel Full access to system

EL2 Hypervisor / Virtual Machine Monitor
Manages virtual machines,
Top-level system control for EL2 (hypervisor / VM)

EL3 Secure Monitor / Firmware Highest privilege, secure world / boot code

Process Context:

When the CPU switches from one process to another, it needs to remember everything about the old process so it can continue later.

This “everything” is called the process context, and it has 3 main parts:

so for example i opened a new process or task that will interrupt another task, the OS context stores the interrupted process details in the PCB to continue it later.

Hardware Context

- Stores the CPU registers of the process so when the process resumes, the CPU can continue exactly where it left off.

System Context

- Stores the OS’s information about the process, such as:
 - Process state (running, ready, waiting), Scheduling info (priority, CPU time used)
 - Resource usage

User Context

- Stores the process’s memory content, including:
 - Code pages, Data pages, Stack pages.
- Basically everything the process needs to run correctly in its virtual memory.

Where it’s stored: in the process control block.

Responsible OS Parts – Dispatcher & Scheduler

Dispatcher:

Actually performs the **context switch**

Moves the CPU from one process to another

Changes process states (e.g., running → waiting, waiting → running)

Scheduler:

Decides which process should run next

Uses a scheduling algorithm (e.g., Round Robin, Priority, FCFS)

It controls the execution order of processes

Why separate Dispatcher from Scheduler?

- **Separation of concerns:**
 - Scheduler = decision maker (strategy, policy)
 - Dispatcher = executor (mechanics of switching)
- **Flexibility:**
 - Can change scheduling algorithm without rewriting the context switch code
- **Clarity:**
 - Easier to design and maintain OS

Process States

each process in a status at any point of time.

The process state tells the OS what the process is doing and how it should be managed.

Ready / Running

- Process is running on the CPU or ready to run
- Example: You start a program, and it's waiting for its turn on the CPU

Waiting / Blocked

- Process is waiting for something (I/O, user input, resources)
- Example: Program waiting for a file to load or a network response

Process State Model lec pt2

Process context:

The process context is everything the operating system must remember about a running program so it can pause it and later resume it correctly.

Hardware context: So the CPU can pause a process and later continue from the exact same spot.

this is the CPU's snapshot of the process. It includes things like:

- Program counter (what instruction to run next)
- CPU registers
- Stack pointer

System context: So the OS can track, schedule, and control the process properly.

This is the OS's own information about the process.

It includes:

- Scheduling info (when it should run)
- Process state (running, ready, waiting...)
- Resources it owns (open files, permissions, etc.)

User: It holds everything the program uses or creates while running.

This is the memory space of the process itself.

It includes:

- Program code
- Global variables/data
- Stack
- Heap

Process Control Block:

This is where the OS finds all the information about a process so it can manage it, a process control block has all the information about a process and many process control blocks form a process control table.

Process control block (PID): a unique number that the OS assigns for a process to identify it.

Parent process identifier (PPID): is the parent process that launched and started your process there is always a parent process (in case of a browser the process that launches your browser when you double click is called browser.exe)

Process State: this describes which state is the process in right now (running, waiting, terminated and so on)

Content of CPU register: This is the saved internal state of the CPU when the process is paused. (stack pointer, program counter) its hardware context.

Open files: the list of files that the current process has opened to run the process (network connections)

Associated peripheral devices: the devices that this current process is using. hardware devices like cameras, mouse and so on.

Consumed CPU time: This is how much CPU time the process has already used.

Allocated Main Memory Areas: These are the memory regions belonging to the process. Code segment, Data segment, Heap. The OS tracks these regions so it knows what memory the process owns.

Priority: This is the importance level of the process in the scheduler.

Process X is switched to Y:

Lets assume process x is running and the OS gets a system call the OS save the current state in the process control block and then calls the process control block of Y until Y process finish then it calls the process control block of X and resume X.

2 Process-State Model:

A process model where we only have 2 states: idle and running first a user creates a process then its assigned a dispatcher and ready to run, then a system call interrupts it and it gets back to the first state.

Pros: Simple

Cons: the assumption is wrong that any process is ready to run at any time; some process are waiting for resources like network, another process to finish or I/O.

3 Process-State Model:

The 3-State Process Model is an improvement over the 2-State model. It recognizes that not all non-running processes are ready to run.

1. Running
 - The process is currently executing on the CPU.
 - Only one process per CPU core can be in this state at a time.
2. Ready
 - The process is not running but is ready to run whenever the CPU is available.
 - Example: A text editor waiting while CPU is free.
3. Waiting (Blocked)
 - The process cannot run yet because it is waiting for some event, e.g.,
 - User input
 - Disk I/O
 - Network data

5 Process-State Model:

Reasons:

RAM limitation: our RAM is limited and in the 3 state model every process that is initiated goes directly to ready, this does not apply since we only have limited resources some process may need to wait for some memory to be free.

Loading from mass storage: Some processes are stored on disk (hard drive/SSD) when created. They cannot run immediately because they must first be loaded into memory.

Added 2 states:

New: Process has just created (PCB created), but not yet admitted to pool of executable processes.

Exit: Process released from pool of executable processes, either halted or aborted for some reason (PCB still exist, ready for deleting).

6 Process-State Model:

Reason 1: Main memory is limited: since the RAM is limited and not all process can fit at the same time so we may move some process to another secondary memory. Process that are suspended or have lower priority.

You open Chrome, VSCode, and a game at the same time. RAM is full, so the OS moves a waiting process, like a background updater, to disk. This frees memory for active processes.

What “Activate” means:

Activate is the OS operation that does the following: Moves the process from secondary memory (disk) to main memory (RAM) Changes its state from Suspended Ready.

In short: “Activate” = bring back from disk so it can run when CPU is free.

7 Process-State Model:

Advantages: More free main memory than in 6 State Model

New (Created) – Process is created, not yet admitted to ready queue.

Ready – Process is in RAM, waiting for CPU.

Running – Process actively using CPU.

Waiting (Blocked) – Process in RAM, waiting for an event.

Ready/Suspended – Process **ready to run** but in **secondary memory**.

Blocked/Suspended – Process **blocked/waiting** and in **secondary memory**.

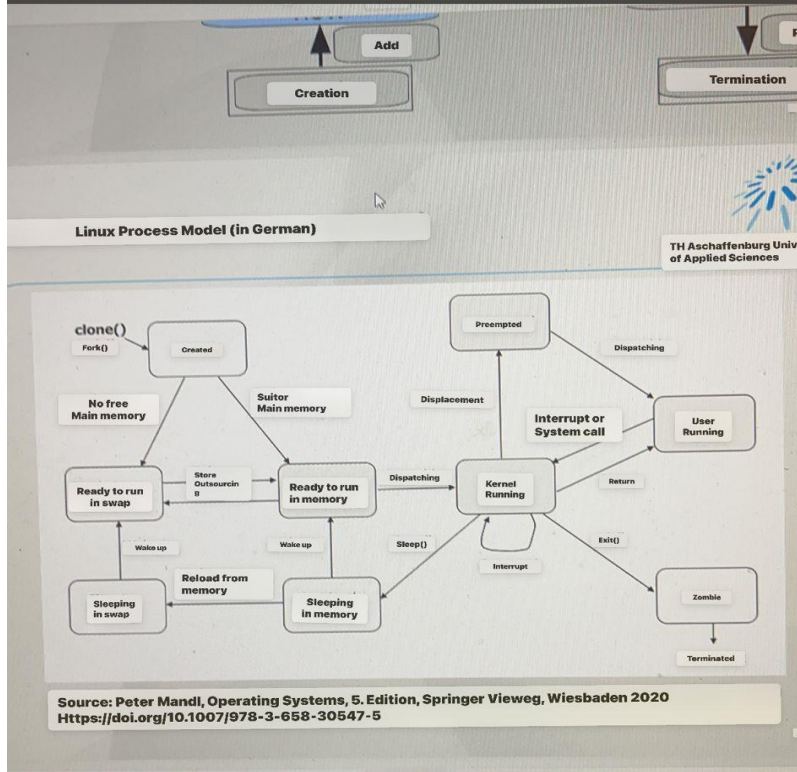
Exit – Process finished, PCB exists for cleanup.

8 Process-State Model:

Adding one new state splitting the running state into 2 states one for user running and another one for kernel running to show different user privileges. This is like a simplified linux OS process state model

N.B: we only switch to kernel state when we get a system call while running in user state, the the kernel state runs back to either user running state or ready state but the kernel running state does not block the process.

23:39



German → English



Translation



Camera



Conversation



Favourites

Process Structure in Memory:

Text → code, instructions and constant variables

BSS → un-initialized/static variables

Data → Initialized/static variables

Heap → dynamic memory grow upwards, grows when you use new() or malloc() and shrinks when you use free() or delete()

Stack → local variables & function calls function parameters and grows downwards

Memory Mapping → shared memory, libraries, files

System calls, shared memory, and file mapping.

argv/env → input arguments and environment stored inside the stack

examples for env: Home="address"

example for argv: input.txt

Example of Process In Memory:

text	data	bss	dec	hex	filename
------	------	-----	-----	-----	----------

2906	656	24	3586	e02	./Debug/8_3_process_in_mem
------	-----	----	------	-----	----------------------------

Size of text in bytes (the program instructions)

Size of data (initialized and static variables)

Size of BSS (uninitialized global and static variables)

Dec is the size of everything in byte

Hex is the equivalent in hex decimal